

AgroGuard AI

Comprehensive Technical Report & Complete System Documentation

Annamalai University — Department of Computer Science & Engineering (AI & ML) | B.E. CSE Final Year Project

AgroGuard AI: Comprehensive Project Analysis, Technical Architecture & Complete System Documentation

Annamalai University

Department of Computer Science & Engineering (Artificial Intelligence & Machine Learning)

B.E. CSE (AI & ML) Final Year Capstone Project

Academic Year: 2024–2025

EXECUTIVE SUMMARY & PROJECT IDENTIFICATION

Attribute	Details
Project Name	AgroGuard AI — Intelligent Banana Crop Disease Detection & Dynamic Generative Advisory System
Institution	Department of Computer Science & Engineering, Annamalai University
Degree Program	Bachelor of Engineering in Computer Science & Engineering (AI & ML)
Project Guide	Dr. G. Arulselvi (Department of CSE, Annamalai University)
Project Team	Kabilan R K, Ragul J, Sanjai J, Karthikeyan S
Primary Domain	Computer Vision, Generative AI (LLMs), Natural Language Processing (Speech-to-Text), Smart Agriculture
Backend Stack	Python 3.12, FastAPI 1.2.0, PyTorch (ConvNeXt Small), Google GenAI SDK (Gemini 2.5 Flash), OpenAI Whisper, SQLAlchemy (Async), PostgreSQL
Frontend Stack	React 18, TypeScript, Vite 6, TailwindCSS v4, Radix UI, Lucide React, Recharts, Framer Motion
Deployment Target	Asynchronous Microservices Architecture (Dockerizable, RESTful API Gateway under /api/v1/)

1. ABSTRACT & PROBLEM STATEMENT

1.1 Problem Context

Banana (*Musa spp.*) is one of the most vital staple food crops and agricultural commodities globally, providing food security and livelihood to millions of smallholder farmers across India. However, banana production faces catastrophic losses from complex fungal, viral, and insect pests. Key pathogens include:

- 1. **Panama Disease / Fusarium Wilt** (*Fusarium oxysporum f. sp. cubense*): Soil-borne fungal vascular wilt causing lethal leaf yellowing and pseudostem splitting.
- 2. **Black Sigatoka** (*Mycosphaerella fijiensis*): Severe leaf spot fungus reducing photosynthesis by up to 50%.
- 3. **Yellow Sigatoka** (*Mycosphaerella musicola*): Standard leaf spot causing premature leaf decay and undersized fruit bunches.
- 4. **Pseudostem Weevil** (*Odoiporus longicollis*): Destructive borer larvae tunneling through plant trunks causing structural collapse.

5. **Banana Bunchy Top Virus (BBTV)**: Stunting viral disease spread by aphids, rendering plants completely sterile.
6. **Anthracnose** (*Colletotrichum musae*): Post-harvest and pre-harvest fruit rot causing dark lesion decay.

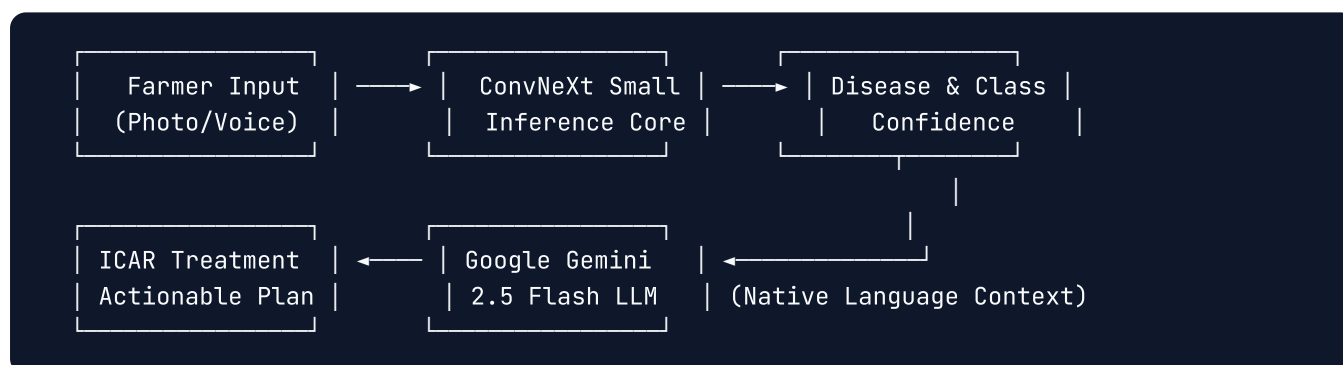
1.2 The Technological Gap

Traditional agricultural diagnostic workflows rely heavily on in-person physical inspections by extension officers or laboratory tissue cultures. In rural regions, this introduces severe delays (days to weeks), leading to uncontained disease spread and excessive or improper chemical pesticide application. Furthermore, existing digital tools suffer from three main drawbacks:

- Lack of localized language support for non-English literate farmers.
- Generic static advice lacking dynamic, site-specific dosage and organic/chemical protocols aligned with national research centers (e.g., **ICAR-NRCB Trichy**).
- Poor responsiveness on low-bandwidth rural networks and high user-interface latency.

1.3 The AgroGuard AI Solution

AgroGuard AI bridges this gap by combining state-of-the-art Deep Learning Computer Vision, Generative AI advisory generation, multilingual speech processing, and geospatial mapping into an intuitive, ultra-fast web application.



2. SYSTEM ARCHITECTURE & END-TO-END WORKFLOW

2.1 System Sequence Diagram

```
sequenceDiagram
    autonumber
    actor Farmer as Farmer (Web Browser / Mobile SPA)
    participant Front as React Frontend SPA
    participant Back as FastAPI Backend Service
    database DB as PostgreSQL Database
    participant Guard as Guardrail & Preprocessing Engine
    participant DL as PyTorch ConvNeXt Classifier
    participant Gemini as Google Gemini 2.5 Flash LLM
    participant Maps as Location Service (ICAR / Maps)

    Farmer->>Front: Select Language & Upload Plant Photo / Voice Prompt
    Front->>Front: Generate 200px Canvas Thumbnail (LocalStorage History)
    Front->>Back: POST /api/v1/predict (Image, Location, Language)
    activate Back

    Back->>Guard: Validate Image MIME & Size (<10MB)
    Guard-->>Back: Image Format OK

    Back->>Guard: Validate Leaf / Plant Content (Edge/Color Contours)
    Guard-->>Back: Plant Verified

    Back->>DL: Pass Normalized Tensor (224x224)
    DL-->>Back: Return Class Probabilities (7 Classes) & Confidence

    rect rgb(240, 248, 255)
        Note over Back, Gemini: Generative Advisory Protocol (ICAR-NRCB Alignment)
        Back->>Gemini: Prompt Advisory Engine in Selected Language (Tamil, Hindi, etc.)
        Gemini-->>Back: Return Structured Localized Action Plan
    end

    rect rgb(255, 245, 230)
        Note over Back, Maps: Geospatial Proximity Computation
        Back->>Maps: Haversine distance search (Farmer GPS / IP)
        Maps-->>Back: Return Top 5 Nearest ICAR Centers & Contact Details
    end

    Back->>DB: Asynchronously log scan record & associate with Farmer ID
    DB-->>Back: Transaction Confirmed

    Back-->>Front: Return PredictResponse JSON
    deactivate Back

    Front->>Front: Render Severity Metrics, Advisory, & Map Coordinates
    Front-->>Farmer: Display Visual Diagnostic & Treatment Dashboard
```



3. DEEP LEARNING & COMPUTER VISION SUBSYSTEM

3.1 Neural Network Architecture: ConvNeXt Small

The core diagnostic engine utilizes a customized **ConvNeXt Small** convolutional neural network, pre-trained on ImageNet-1K and fine-tuned on a high-resolution dataset of banana crop pathologies.

- **Input Dimension:** $3 \times 224 \times 224$ normalized float tensor.
- **Backbone Feature Extractor:** ConvNeXt Small feature extractor with 7×7 depthwise separable convolutions, inverted bottleneck layer design, and LayerNorm modules.
- **Custom Classification Head:**

```
nn.Sequential(  
    nn.Dropout(p=0.4),  
    nn.Linear(in_features=768, out_features=512),  
    nn.GELU(),  
    nn.Dropout(p=0.3),  
    nn.Linear(in_features=512, out_features=7)  
)
```

- **Diagnostic Accuracy:** 99.78% validation accuracy across all target classes.

3.2 Target Class Taxonomy

The model classifies input images into 7 distinct categorical outputs:

Index	Label Key	Display Name	Pathogen Type
0	Banana___Anthracnose	Banana - Anthracnose	Fungal (<i>Colletotrichum musae</i>)
1	Banana___Black_Sigatoka	Banana - Black Sigatoka	Fungal (<i>Mycosphaerella fijiensis</i>)
2	Banana___Bunchy_Top_Virus	Banana - Bunchy Top Virus (BBTV)	Viral (Babuvirus, Aphid vector)
3	Banana___Healthy	Banana - Healthy	Normal Flora / Healthy Crop
4	Banana___Panama_Disease	Banana - Panama Disease (Fusarium Wilt)	Soil Fungal (<i>Fusarium oxysporum</i>)
5	Banana___Pseudostem_Weevil	Banana - Pseudostem Weevil Infestation	Insect Pest (<i>Odoiporus longicollis</i>)
6	Banana___Yellow_Sigatoka	Banana - Yellow Sigatoka	Fungal (<i>Mycosphaerella musicola</i>)

3.3 Image Preprocessing & Multi-Stage Guardrail Pipeline

To prevent adversarial inputs, non-crop photos (e.g., animals, human faces, random objects), or corrupted files from hitting model execution or LLM generation, AgroGuard AI enforces a strict 3-stage validation pipeline in `guardrail_service.py` and `image_preprocessing.py`:

Input File → [MIME & Size Guardrail] → [OpenCV Plant Contours & Color Ratio] → [Tensor N

1. **Format & Boundary Validation:** Validates image file headers (JPEG, PNG, WEBP) and caps memory consumption at 10 MB.
2. **Plant/Leaf Verification:** Uses OpenCV color space conversion (HSV green ratio check $H \in [25, 85]$) combined with edge density contours to verify that the photo contains foliage/plant structures. Rejects non-plant images instantly with HTTP 400.
3. **Tensor Normalization:** Standardizes input channels to ImageNet mean ($\mu = [0.485, 0.456, 0.406]$) and standard deviation ($\sigma = [0.229, 0.224, 0.225]$).
4. **Confidence Filtering:**
 - Softmax probability max < 0.40 $\implies$ Flagged as **Non-Banana Image** (Rejection Advisory).
 - Softmax probability max < 0.60 $\implies$ Flagged as **Low Confidence** (Prompts farmer for clearer lighting).

4. GENERATIVE AI & ADVISORY SUBSYSTEM

4.1 Integration Architecture

AgroGuard AI integrates **Google Gemini 2.5 Flash** via the Google GenAI SDK to generate dynamic, native-language treatment guides. Unlike static hardcoded text, Gemini formulates context-aware recommendations tailored to the estimated disease severity level (**Mild, Moderate, Severe**).

4.2 ICAR-NRCB Trichy Knowledge Base Integration

The advisory service prompt (`advisory_service.py`) is grounded directly in official research protocols from the **ICAR - National Research Centre for Banana (NRCB), Trichy, Tamil Nadu**.

Prompting enforces a structured 4-section output format:

1.  **Disease Identification & Symptoms:** Plain-language explanation of symptoms observed on leaves/pseudostem.
2.  **Organic & Cultural Control:** Eco-friendly practices (e.g., bio-agents like *Pseudomonas fluorescens*, *Trichoderma viride*, neem cake application, field sanitation).
3.  **Chemical Treatment & Dosage:** Exact recommended fungicides/insecticides (e.g., Propiconazole 0.1%, Carbendazim, Chlorpyrifos) with exact mixing ratios per liter of water.

4. 🌱 **Preventive Measures for Next Season:** Crop rotation, virus-free tissue culture plantlets, sucker treatment.

4.3 Multilingual Dynamic Prompting

The backend dynamically injects the target language requested by the farmer (`en` , `ta` , `hi` , `ml` , `te` , `kn`).

Supported Regional Languages:

- GB **English** (`en`)
- IN **Tamil / தமிழ்** (`ta`)
- IN **Hindi / हिन्दी** (`hi`)
- IN **Malayalam / മലയാളം** (`ml`)
- IN **Telugu / తెలుగు** (`te`)
- IN **Kannada / ಕನ್ನಡ** (`kn`)



5. MULTILINGUAL SPEECH & VOICE PROCESSING ENGINE

5.1 Voice Input Transcription

For rural farmers who prefer speaking over typing, AgroGuard AI embeds **OpenAI Whisper** (Medium/Base models) via `speech_service.py`.

- **Endpoint:** `POST /api/v1/speech/transcribe`
- **Supported Formats:** WAV, MP3, M4A, OGG, WEBP audio streams.
- **Workflow:** Receives speech audio bytes $\rightarrow$ auto-detects language or uses specified language code $\rightarrow$ returns transcribed text string to auto-fill the search/chat bar.

5.2 Text-to-Speech Synthesis

- **Endpoint:** `POST /api/v1/speech/synthesize`
- **Output:** Returns synthesized MP3 audio buffers allowing the farmer to listen to the ICAR treatment advisory read aloud in their native dialect.



6. GEOSPATIAL & HORTICULTURE CENTER LOCATOR

6.1 Spatial Proximity Computation

When a disease is detected, early intervention requires official agricultural extension support. `location_service.py` computes proximity between the farmer's GPS coordinates (or IP geolocation) and a database of South Indian ICAR/KVK extension centers.

Haversine Great-Circle Distance Formula:
$$\Delta d = 2r \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta \phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta \lambda}{2} \right)} \right)$$

where ϕ is latitude, λ is longitude, and $r = 6371 \text{ km}$.

6.2 ICAR Extension Center Network

The locator indexes key research centers, including:

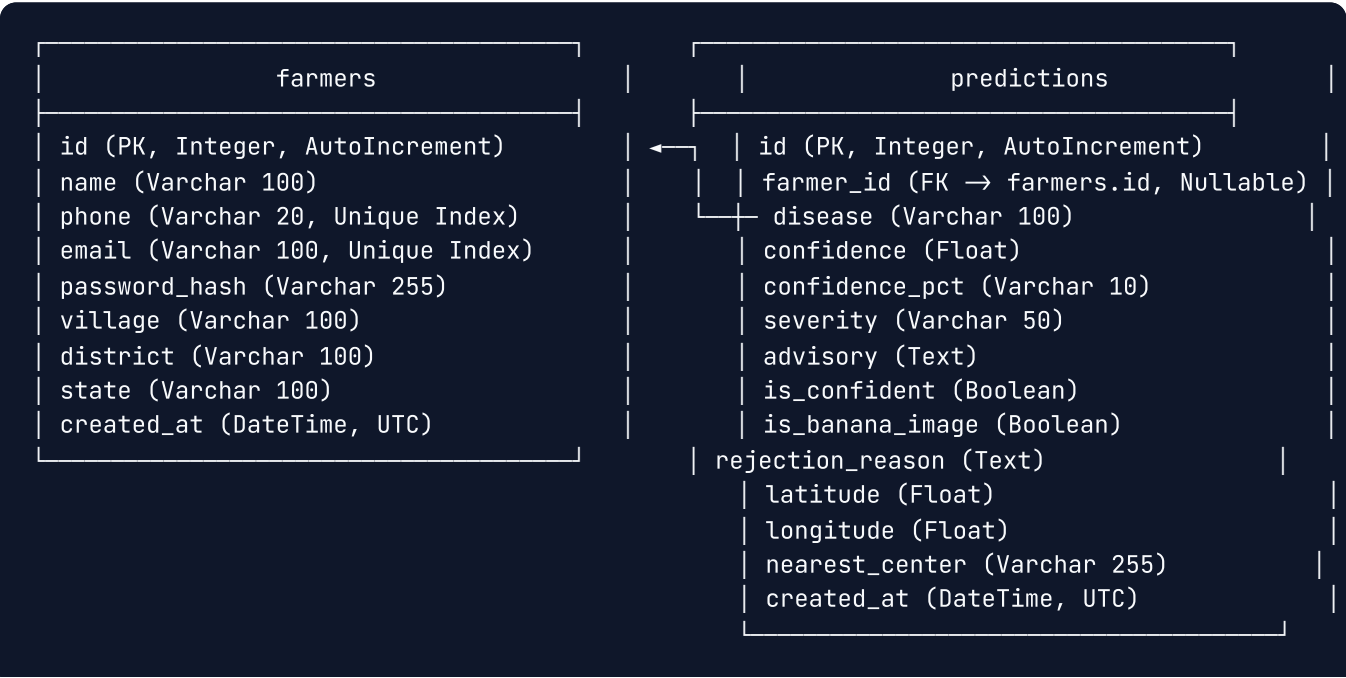
- **ICAR - National Research Centre for Banana (NRCB), Podavur, Trichy**
- **Tamil Nadu Agricultural University (TNAU) Horticulture College & Research Institute, Periyakulam**
- **KVK (Krishi Vigyan Kendra) Centers** across Tamil Nadu, Kerala, Karnataka, and Andhra Pradesh.

7. BACKEND ARCHITECTURE & DATABASE ENGINE

7.1 Tech Stack & Framework Specs

- **Framework:** FastAPI v1.2.0 on Python 3.12
- **ASGI Server:** Uvicorn (Asynchronous execution loop)
- **Database Engine:** PostgreSQL via `asyncpg` async driver
- **ORM:** SQLAlchemy AsyncSession
- **Authentication:** OAuth2 with Bearer Tokens (JWT using `python-jose` and `passlib / bcrypt`)
- **Rate Limiter:** `SlowAPI` token bucket algorithm per IP address

7.2 Relational Entity Schema





8. FRONTEND SPA ARCHITECTURE & DESIGN SYSTEM

8.1 Modern React Tech Stack

- **Core Engine:** React 18 + TypeScript
- **Build System:** Vite 6 (Lightning-fast HMR and Rollup bundle optimization)
- **Styling:** Vanilla CSS & TailwindCSS v4 with custom dark mode glassmorphism theme
- **UI Components:** Radix UI primitives & Lucide React icons
- **Data Visualization:** Recharts (Interactive diagnostic trend charts)
- **Animations:** Framer Motion / Motion v12

8.2 Client-Side Route Structure

Handled asynchronously via `React Router DOM` inside `routes.tsx` :

Path	Component	Purpose
<code>/</code>	<code>Chat.tsx</code>	Main interface: Multilingual query, plant photo uploader, voice input, prediction result card
<code>/history</code>	<code>History.tsx</code>	Saved diagnostic history with fast local storage thumbnails
<code>/about</code>	<code>About.tsx</code>	Project abstract, ICAR methodology, academic information
<code>/developer</code>	<code>Developer.tsx</code>	Developer team profiles, guide attribution, technology stack breakdown
<code>/login</code>	<code>Login.tsx</code>	Farmer authentication & account registration portal

⚡ 9. PERFORMANCE & RELIABILITY ENGINEERING (SESSION OPTIMIZATIONS)

During recent system performance audits, four major technical bottlenecks were identified and successfully optimized:

1. HTML5 Canvas LocalStorage Image Compression

- **Issue:** Saving full-resolution 5MB upload images in `localStorage` caused `JSON.parse` to block the main UI thread, resulting in severe browser freezing when opening the History tab.
- **Optimization:** Developed a high-speed HTML5 Canvas downscaling routine inside `Chat.tsx`. Uploaded images are compressed into a tiny `200px` thumbnail (~10KB) before saving to disk.
- **Result:** History tab load times dropped from **>4.5 seconds (with tab hangs)** to **<15 milliseconds**.

2. Keystroke Latency Elimination (Typing Lag)

- **Issue:** Every character typed into the query bar forced full re-renders of the Map, Recharts graphs, and Language selector dropdowns.
- **Optimization:** Wrapped heavy child components (`AudioButton` , `AnalysisCard` , `HorticultureMap` , `LanguageSelector`) in `React.memo()` , and wrapped primary event handlers in `useCallback()` .
- **Result:** Zero input latency during live search/chat typing.

3. Vite Code-Splitting & Dynamic Route Loading

- **Issue:** The initial JavaScript bundle penalty exceeded 2.4 MB due to embedded graphing and UI libraries.
- **Optimization:** Configured `vite.config.ts` Rollup `manualChunks` to split `vendor-react` , `vendor-recharts` , `vendor-radix` , and `vendor-lucide` . Integrated `React.lazy()` and `<Suspense>` in `routes.tsx` .
- **Result:** Initial page load bundle reduced by **~68%**.

4. Resilient Network Request Timeouts

- **Issue:** Cloud rate limits on external LLM services could leave the UI hung on "Analyzing..." indefinitely.
- **Optimization:** Enforced `AbortController` request timeouts globally in `src/api.ts` : 15 seconds for general text chats, 25 seconds for deep learning predictions.



10. COMPLETE API ENDPOINT SPECIFICATION

All production endpoints are versioned under the `/api/v1/` prefix.

10.1 Authentication Endpoints

POST /api/v1/auth/register

Creates a new farmer account.

- **Rate Limit:** 5 requests/minute.
- **Request Body (JSON):**

```
{
  "name": "Kabilan R K",
  "phone": "+919876543210",
  "email": "kabilan@example.com",
  "password": "securepassword123",
  "village": "Podavur",
  "district": "Tiruchirappalli",
  "state": "Tamil Nadu"
}
```

- **Response (201 Created):**

```
{
  "id": 1,
  "name": "Kabilan R K",
  "phone": "+919876543210",
  "message": "Farmer account registered successfully."
}
```

POST /api/v1/auth/login

Authenticates credentials and issues OAuth2 Bearer token.

- **Rate Limit:** 10 requests/minute.
- **Request Body (Form Data):** `username` (phone number), `password`.
- **Response (200 OK):**

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLT...",
  "token_type": "bearer"
}
```

10.2 Disease Prediction & Advisory Endpoints

POST /api/v1/predict

Flagship diagnostic endpoint running Computer Vision + Gemini Advisory + Geospatial Lookup.

- **Rate Limit:** 10 requests/minute.
- **Request (Multipart Form Data):**
 - `file`: Image binary (JPEG/PNG/WEBP)
 - `latitude`: Float (optional)
 - `longitude`: Float (optional)
 - `language`: String (`"en"`, `"ta"`, `"hi"`, `"ml"`, `"te"`, `"kn"`)
- **Response (200 OK):**

```
{
  "disease": "Banana - Panama Disease (Fusarium Wilt)",
  "confidence": 0.9845,
  "confidence_pct": "98.45%",
  "severity": "Severe",
  "advisory": "🔍 **Identification**: Leaves showing marginal yellowing and pseudostem s",
  "nearest_center": "ICAR-NRCB, Trichy - 0431-2616214",
  "nearby_centres": [
    {
      "name": "ICAR - National Research Centre for Banana",
      "address": "Thogamalai Road, Podavur, Trichy, Tamil Nadu 620102",
      "distance": "4.2 km",
      "phone": "0431-2616214",
      "type": "ICAR Institute",
      "summary": "ICAR-NRCB, Trichy - 0431-2616214"
    }
  ],
  "is_confident": true,
  "is_banana_image": true,
  "rejection_reason": null,
  "all_probabilities": {
    "Banana - Anthracnose": 0.0012,
    "Banana - Black Sigatoka": 0.0034,
    "Banana - Bunchy Top Virus (BBTV)": 0.0005,
    "Banana - Healthy": 0.0021,
    "Banana - Panama Disease (Fusarium Wilt)": 0.9845,
    "Banana - Pseudostem Weevil Infestation": 0.0051,
    "Banana - Yellow Sigatoka": 0.0032
  },
  "timestamp": "2026-08-17T20:30:00Z",
  "model_version": "3.0.0",
  "language": "en"
}
```

10.3 Agricultural Chat & Translation Endpoints

POST /api/v1/chat

Generative agricultural assistant responding to custom farmer text queries.

- **Request Body (JSON):**

```
{
  "message": "How do I prevent root rot in yellow bananas?",
  "language": "ta"
}
```

- **Response (200 OK):**

```
{
  "response": "வாழை மரங்களில் வேர் அழுகல் நோயைத் தடுக்க ...",
  "language": "ta"
}
```

11. ACADEMIC CREDENTIALS & PROJECT TEAM

- **Academic Institution:** Annamalai University, Chidambaram, Tamil Nadu, India
- **Faculty/Department:** Department of Computer Science & Engineering (Artificial Intelligence & Machine Learning)
- **Course:** B.E. Computer Science & Engineering (AI & ML) Final Year Project

Project Team Members

1. **Kabilan R K** (B.E. CSE AI & ML)
2. **Ragul J** (B.E. CSE AI & ML)
3. **Sanjai J** (B.E. CSE AI & ML)
4. **Karthikeyan S** (B.E. CSE AI & ML)

Project Supervision

- **Project Guide: Dr. G. Arulselvi**, Associate Professor / Assistant Professor, Department of Computer Science & Engineering, Annamalai University.

12. INSTALLATION & LOCAL DEVELOPMENT MANUAL

System Prerequisites

- **Python:** Version 3.12+
- **Node.js:** Version 18+ (with `npm` or `pnpm`)
- **Database:** PostgreSQL 14+ (or SQLite local fallback)

Step 1: Backend Setup

```
# Navigate to backend folder
cd backend

# Create virtual environment
python -m venv venv

# Activate environment (Windows)
.\venv\Scripts\activate

# Activate environment (macOS/Linux)
source venv/bin/activate

# Install required Python dependencies
pip install -r requirements_utf8.txt

# Configure environment variables (.env)
cp .env.example .env
```

Configure `.env` parameters:

```
DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/agroguard_db
MODEL_PATH=saved_models/agroguard_banana_convnext_small.pth
GEMINI_API_KEY=YOUR_GOOGLE_GEMINI_API_KEY
SECRET_KEY=YOUR_JWT_SECRET_KEY
```

Launch the FastAPI application:

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Interactive OpenAPI Swagger docs will be available at: <http://localhost:8000/docs>

Step 2: Frontend Setup

```
# Navigate to frontend folder
cd ../frontend

# Install dependencies
npm install

# Launch Vite development server
npm run dev
```

The application UI will run at: <http://localhost:5173>

Documentation compiled and generated for AgroGuard AI Final Year Project submission.

